

2.3 DynamoDB and NoSQL

i This is our main database. It works nothing like SQL, and the difference matters enormously. Getting a table design wrong is expensive to undo later, because you can't just add a join.

This task/page is bigger than any of the previous tasks you have been doing because it is really teaching you **database design first, coding second**.

Learning objective

What does NoSQL mean, and what is DynamoDB good and bad at?

DEF: NoSQL means "not only SQL" or "non-relational". It is a type of database that stores and retrieves data without using traditional tables, rows, and columns. Instead, it uses flexible formats like documents, key-value pairs, or graphs, making it great for fast, large-scale, and unstructured data.

Partition key vs sort key — and why you have to decide these up front

Partition keys are ways to group information in a database. They are used to design how data is organised across the database.

Sort keys are ways to 'sort' the data to give you a more accurate result when trying to pull certain information out of a database. For example, you could sort the data by date and time, resulting in the earlier information input in a partition group being displayed first, making finding your result easier.

What is an "access pattern", and why do you design the table around your queries rather than around your data?

An access pattern is the exact way your app reads and writes data. It details whether you need a single item, a sorted list, or a bulk update.

{{query}} vs {{scan}} — and why scan is almost always the wrong answer

A query uses an index to jump straight to the exact data you need. A scan reads every single row or item in a table from start to finish. Scan is usually the wrong answer because it wastes massive amounts of compute time, memory, and money as data grows.

What is a Global Secondary Index for?

It is a database index that contains a completely separate partition and sort key.

DEF: A DynamoDB Global Secondary Index (GSI) is a type of secondary index that allows you to query a table using a different partition key and optionally a sort key from those defined in the table's primary key. GSIs give you powerful flexibility to support multiple access patterns without duplicating data across tables. All GSI reads are eventually consistent; strongly consistent reads are not supported.

Task

- i**
- Do
 - Create your table, named with your prefix, e.g. {{yourname-coldchain-readings}}

- From your Lambda: write a reading, read it back by its keys, run a query for a branch's readings, run a query for a date range, update one, delete one
- *Done when*
 - All of those operations work from your Lambda
 - Your key design and the queries it supports are written into the worksheet, with your reasoning
 - You can explain why you chose that partition key, and what would break if you'd chosen `{{readingId}}` instead
 - Access patterns 1, 2 and 3 all work with `{{query}}`, not `{{scan}}`

The order of working that I have laid out

Understand the access patterns → design the keys → create the table → give Lambda IAM permission → connect Lambda to DynamoDB → test create/read/query/update/delete → document the reasoning.

- ✓ Understand the access patterns
- ✓ design the keys
- ✓ create the table
- ✓ give Lambda IAM permission
- ✓ coding index.jms lambda function
- ✓ test create/read/query/update/delete
- ✓ document the reasoning.

Understand the access patterns

So we have to be building the database around these four concepts

1. All readings for one branch, newest first
2. All readings for one branch between two dates
3. One specific reading
4. Stretch: all excursions across all branches

The first three are linked to 'branches' — that is a clue for your partition key; therefore I would name the partition key `branchId`. Simply because all a single branch's data (fridges) can be put under one `branchId` and be called easily.

NOTE: a **partition key** in DynamoDB is basically the main value AWS uses to decide where an item belongs and which group of items to look in. E.G.

If your partition key is:

```
1 | branchId
```

then all readings for the same branch are grouped under the same branch value.

```
1 | BRANCH-001
2 |   - Reading A
3 |   - Reading B
4 |   - Reading C
5 |
6 | BRANCH-002
```

```
7 - Reading D
8 - Reading E
```

So when you ask DynamoDB, "Give me all readings for BRANCH-001" it knows exactly which group to look at.

NOTE: a **sort key** is the second part of a DynamoDB key that helps organise items inside one partition. E.G.

```
1 | BRANCH-001
2 |   2026-08-10T09:00:00Z#READING-001
3 |   2026-08-11T09:00:00Z#READING-002
4 |   2026-08-12T09:00:00Z#READING-003
```

Here, **BRANCH-001** groups the readings together, and the sort key decides their order inside that group.

Because your sort key starts with the date/time, DynamoDB can easily do things like:

(Give me all readings for BRANCH-001) or (Give me readings for BRANCH-001 between 10 August and 12 August) or (Give me the newest readings first)

The simplest way to remember it is:

Partition key = which group does this item belong to?

Sort key = how are the items inside that group organised and identified?

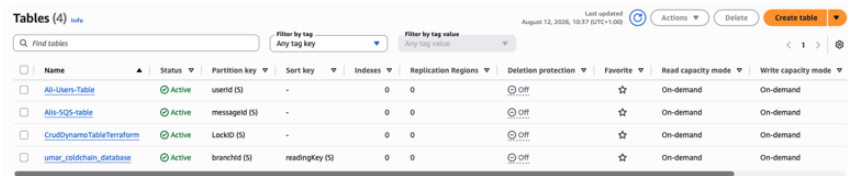
Design keys and creating table

So as a result I would build the database as follows below:

Table name = umar-coldchain-database

Partition key = branchId

Sort key = readingKey



Name	Status	Partition key	Sort key	Indexes	Replication Regions	Deletion protection	Favorite	Read capacity mode	Write capacity mode
Ali-Users-Table	Active	userId (S)	-	0	0	Off	☆	On-demand	On-demand
Alix-SQS-table	Active	messageId (S)	-	0	0	Off	☆	On-demand	On-demand
CrossDynamoTableTransform	Active	LockId (S)	-	0	0	Off	☆	On-demand	On-demand
umar_coldchain_database	Active	branchId (S)	readingKey (S)	0	0	Off	☆	On-demand	On-demand

Okay, now that the table and keys are built I need to give the Lambda access to my DynamoDB.

Give Lambda IAM permission

I went to the Lambda's configuration then permissions → clicked on the link to the execution role → Create inline policy → then went to the JSON tab and pasted the following code in

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
```

```

7     "dynamodb:PutItem",
8     "dynamodb:GetItem",
9     "dynamodb:Query",
10    "dynamodb:UpdateItem",
11    "dynamodb:DeleteItem"
12  ],
13  "Resource": "arn:aws:dynamodb:eu-north-
1:194442925473:table/umar-coldchain-readings"
14  }
15  ]
16  }

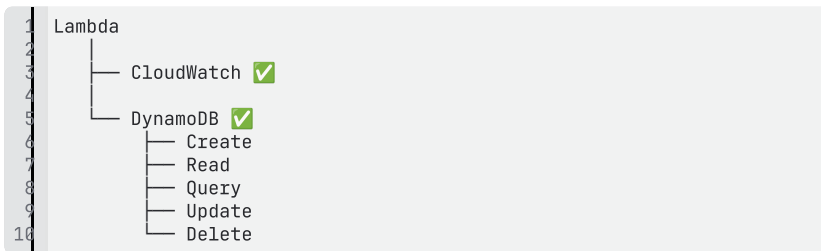
```

Allowed the Lambda to have access and do the CRUD HTTP functions to the DynamoDB database.

Name of new policy = ColdChainDynamoDBAccess

Why have we created this

After adding the policy:



Without these IAM permissions, your JavaScript could be completely correct and AWS would still return an **AccessDenied** error when it tried to access DynamoDB.

 **Extra configuration:** I also used environment variables because the name for the database was extremely long, so I added a key for it called TABLE_NAME. That way, whenever I write TABLE_NAME it will hold the value of the database's actual name.

Environment variables (1)

The environment variables below are encrypted at rest with the default Lambda service key.

Key	Value
TABLE_NAME	umar-coldchain-database

It also keeps configuration separate from your code.

Coding index.js lambda function

Okay, now we have to create the JavaScript that will go into the Lambda index.js. (I used AI to write much of this code; it was simpler than writing the rest by hand in these tasks.)

```

1  import { DynamoDBClient } from "@aws-sdk/client-dynamodb";
2
3  import {
4    DynamoDBDocumentClient,
5    PutCommand,
6    GetCommand,
7    QueryCommand,
8    UpdateCommand,
9    DeleteCommand
10 } from "@aws-sdk/lib-dynamodb";
11
12

```

```

13 const client = new DynamoDBClient({});
14 const dynamo = DynamoDBDocumentClient.from(client);
15
16 const TABLE_NAME = process.env.TABLE_NAME;
17
18
19 function getReadingStatus(minTempC, maxTempC) {
20
21   if (!Number.isFinite(minTempC) || !Number.isFinite(maxTempC)) {
22     throw new TypeError(
23       "minTempC and maxTempC must both be valid numbers"
24     );
25   }
26
27   if (minTempC < 2 || maxTempC > 8) {
28     return "excursion";
29   }
30
31   return "ok";
32 }
33
34
35 export const handler = async (event, context) => {
36
37   console.log("Request received", {
38     requestId: context.awsRequestId,
39     action: event.action,
40     branchId: event.branchId
41   });
42
43
44   switch (event.action) {
45
46     // CREATE A READING
47     case "create": {
48       const status = getReadingStatus(
49         event.minTempC,
50         event.maxTempC
51       );
52
53       const readingKey =
54         `${event.recordedAt}#${event.readingId}`;
55
56       const item = {
57         branchId: event.branchId,
58         readingKey: readingKey,
59         readingId: event.readingId,
60         recordedAt: event.recordedAt,
61         minTempC: event.minTempC,
62         maxTempC: event.maxTempC,
63         recordedBy: event.recordedBy,
64         status: status
65       };
66
67       await dynamo.send(
68         new PutCommand({
69           TableName: TABLE_NAME,
70           Item: item
71         })
72       );
73
74       return {
75         message: "Reading created",
76         item: item
77       };
78     }
79
80
81     // READ ONE SPECIFIC READING
82     case "get": {
83       const result = await dynamo.send(
84         new GetCommand({
85           TableName: TABLE_NAME,
86           Key: {
87             branchId: event.branchId,
88             readingKey: event.readingKey
89           }
90         })
91       );
92
93       return {
94         item: result.Item
95       };

```

```

96     }
97
98
99     // QUERY EVERY READING FOR ONE BRANCH
100    case "queryBranch": {
101        const result = await dynamo.send(
102            new QueryCommand({
103                TableName: TABLE_NAME,
104
105                KeyConditionExpression:
106                    "branchId = :branchId",
107
108                ExpressionAttributeValues: {
109                    ":branchId": event.branchId
110                },
111
112                ScanIndexForward: false
113            })
114        );
115
116        return {
117            readings: result.Items
118        };
119    }
120
121
122    // QUERY READINGS BETWEEN TWO DATES
123    case "queryDateRange": {
124        const result = await dynamo.send(
125            new QueryCommand({
126                TableName: TABLE_NAME,
127
128                KeyConditionExpression:
129                    "branchId = :branchId AND readingKey BETWEEN :from AND
130                    :to",
131
132                ExpressionAttributeValues: {
133                    ":branchId": event.branchId,
134                    ":from": `${event.from}#`,
135                    ":to": `${event.to}#\uffff`
136                },
137
138                ScanIndexForward: false
139            })
140        );
141
142        return {
143            readings: result.Items
144        };
145    }
146
147    // UPDATE A READING
148    case "update": {
149        const result = await dynamo.send(
150            new UpdateCommand({
151                TableName: TABLE_NAME,
152
153                Key: {
154                    branchId: event.branchId,
155                    readingKey: event.readingKey
156                },
157
158                UpdateExpression:
159                    "SET recordedBy = :recordedBy",
160
161                ExpressionAttributeValues: {
162                    ":recordedBy": event.recordedBy
163                },
164
165                ReturnValues: "ALL_NEW"
166            })
167        );
168
169        return {
170            message: "Reading updated",
171            item: result.Attributes
172        };
173    }
174
175
176    // DELETE A READING
177    case "delete": {

```

```

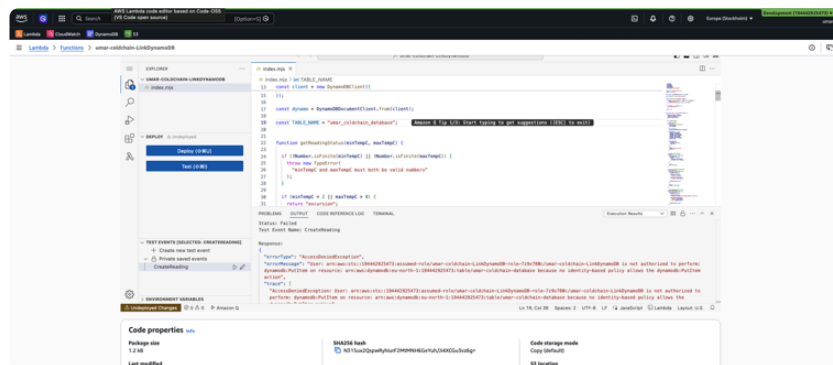
178     await dynamo.send(
179       new DeleteCommand({
180         TableName: TABLE_NAME,
181
182         Key: {
183           branchId: event.branchId,
184           readingKey: event.readingKey
185         }
186       })
187     );
188
189     return {
190       message: "Reading deleted"
191     };
192   }
193
194   default:
195     throw new Error(
196       "Invalid action. Use create, get, queryBranch,
197       queryDateRange, update or delete."
198     );
199   }
200 };

```

It is a lot of code but we tested / ran it in chunks. This is because there are a lot of different functions within this handler.

Test create/read/query/update/delete

First I had loads of errors I needed to iron out before I could start testing. The main issue stemmed from these issues that kept popping up.



The Lambda could not access the database: the issue was completely due to the naming of the DynamoDB table. First issue was when I gave permissions to the Lambda I used this JSON code below:

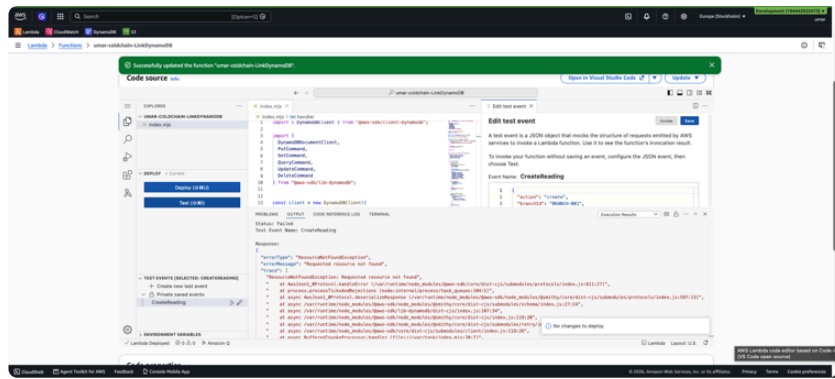
```

1  {
2    "Version": "2012-10-17",
3    "Statement": [
4      {
5        "Effect": "Allow",
6        "Action": [
7          "dynamodb:PutItem",
8          "dynamodb:GetItem",
9          "dynamodb:Query",
10         "dynamodb:UpdateItem",
11         "dynamodb:DeleteItem"
12       ],
13       "Resource": "arn:aws:dynamodb:eu-north-
14         1:194442925473:table/umar-coldchain-readings"
15     }
16   ]
17 }

```

The issue was the name of the database was umar-coldchain-readings when it should have been umar-coldchain-database.

After this, a new error appeared saying the table wasn't in the region.



So it led to an hour of me changing the code around, then using Cloud Shell to run commands to see which DynamoDB tables were in the region, and only then did I realise that the database 'umar-coldchain-database' was actually named 'umar_coldchain_database' with an underscore instead of a hyphen. The image below shows me using Cloud Shell commands:



As a result, the code ended up changing to this:

First issue was when I gave permissions to the Lambda I used this JSON code below:

```

1  import { DynamoDBClient } from "@aws-sdk/client-dynamodb";
2
3  import {
4    DynamoDBDocumentClient,
5    PutCommand,
6    GetCommand,
7    QueryCommand,
8    UpdateCommand,
9    DeleteCommand
10 } from "@aws-sdk/lib-dynamodb";
11
12
13 const client = new DynamoDBClient({
14   region: "eu-north-1"
15 });
16
17 const dynamo = DynamoDBDocumentClient.from(client);
18
19 const TABLE_NAME = "umar_coldchain_database";
20
21
22 function getReadingStatus(minTempC, maxTempC) {
23
24   if (!Number.isFinite(minTempC) || !Number.isFinite(maxTempC)) {
25     throw new TypeError(
26       "minTempC and maxTempC must both be valid numbers"
27     );
28   }
29
30   if (minTempC < 2 || maxTempC > 8) {
31     return "excursion";
32   }
33
34   return "ok";
35 }
36
37
38 export const handler = async (event, context) => {
39   console.log("Request received", {
40

```

```

41     requestId: context.awsRequestId,
42     action: event.action,
43     branchId: event.branchId
44   });
45
46   switch (event.action) {
47     // CREATE A READING
48     case "create": {
49       const status = getReadingStatus(
50         event.minTempC,
51         event.maxTempC
52       );
53
54       const readingKey =
55         `${event.recordedAt}#${event.readingId}`;
56
57       const item = {
58         branchId: event.branchId,
59         readingKey: readingKey,
60         readingId: event.readingId,
61         recordedAt: event.recordedAt,
62         minTempC: event.minTempC,
63         maxTempC: event.maxTempC,
64         recordedBy: event.recordedBy,
65         status: status
66       };
67
68       await dynamo.send(
69         new PutCommand({
70           TableName: TABLE_NAME,
71           Item: item
72         })
73       );
74
75       return {
76         message: "Reading created",
77         item: item
78       };
79     }
80
81     // READ ONE SPECIFIC READING
82     case "get": {
83       const result = await dynamo.send(
84         new GetCommand({
85           TableName: TABLE_NAME,
86           Key: {
87             branchId: event.branchId,
88             readingKey: event.readingKey
89           }
90         })
91       );
92
93       return {
94         item: result.Item
95       };
96     }
97
98     // QUERY EVERY READING FOR ONE BRANCH
99     case "queryBranch": {
100       const result = await dynamo.send(
101         new QueryCommand({
102           TableName: TABLE_NAME,
103
104           KeyConditionExpression:
105             "branchId = :branchId",
106
107           ExpressionAttributeValues: {
108             ":branchId": event.branchId
109           },
110
111           ScanIndexForward: false
112         })
113       );
114
115       return {
116         readings: result.Items
117       };
118     }
119   }
120 }
121
122
123

```

```

124
125 // QUERY READINGS BETWEEN TWO DATES
126 case "queryDateRange": {
127     const result = await dynamo.send(
128         new QueryCommand({
129             TableName: TABLE_NAME,
130
131             KeyConditionExpression:
132                 "branchId = :branchId AND readingKey BETWEEN :from AND
133 :to",
134
135             ExpressionAttributeValues: {
136                 ":branchId": event.branchId,
137                 ":from": `${event.from}#`,
138                 ":to": `${event.to}#\uffff`
139             },
140
141             ScanIndexForward: false
142         })
143     );
144     return {
145         readings: result.Items
146     };
147 }
148
149 // UPDATE A READING
150 case "update": {
151     const result = await dynamo.send(
152         new UpdateCommand({
153             TableName: TABLE_NAME,
154
155             Key: {
156                 branchId: event.branchId,
157                 readingKey: event.readingKey
158             },
159
160             UpdateExpression:
161                 "SET recordedBy = :recordedBy",
162
163             ExpressionAttributeValues: {
164                 ":recordedBy": event.recordedBy
165             },
166
167             ReturnValues: "ALL_NEW"
168         })
169     );
170     return {
171         message: "Reading updated",
172         item: result.Attributes
173     };
174 }
175
176 // DELETE A READING
177 case "delete": {
178     await dynamo.send(
179         new DeleteCommand({
180             TableName: TABLE_NAME,
181
182             Key: {
183                 branchId: event.branchId,
184                 readingKey: event.readingKey
185             }
186         })
187     );
188     return {
189         message: "Reading deleted"
190     };
191 }
192
193 default:
194     throw new Error(
195         "Invalid action. Use create, get, queryBranch,
196 queryDateRange, update or delete."
197     );
198 }
199 };

```

And the JSON permission code ends up changing to this:

```
1 {
2   "Version": "2012-10-17",
3   "Statement": [
4     {
5       "Effect": "Allow",
6       "Action": [
7         "dynamodb:PutItem",
8         "dynamodb:GetItem",
9         "dynamodb:Query",
10        "dynamodb:UpdateItem",
11        "dynamodb>DeleteItem"
12      ],
13      "Resource": "arn:aws:dynamodb:eu-north-
14        1:194442925473:table/umar_coldchain_database"
15    }
16  ]
17 }
```

Then we started the tests

First test: WRITE a reading

Test script: **CreateReading**

```
1 {
2   "action" : "create",
3   "branchId" : "BRANCH-001",
4   "readingId" : "READING-001",
5   "recordedAt" : "2026-08-12T09:00:00.000Z",
6   "minTempC" : 3,
7   "maxTempC" : 7,
8   "recordedBy" : "Umar"
9 }
```

Result = this will add a physical reading (value) into the DynamoDB.

READ the same item back

Test script: **GetReading**

```
1 {
2   "action" : "get",
3   "branchId" : "BRANCH-001",
4   "readingKey" : "2026-08-12T09:00:00.000Z#READING-001"
5 }
```

Result = this is reading back your current reading (value) in the database from just the keys. Result:

```
{
  "item": {
    "recordedBy": "Umar",
    "readingId": "READING-001",
    "status": "ok",
    "minTempC": 3,
    "maxTempC": 7,
    "recordedAt": "2026-08-12T09:00:00.000Z",
    "branchId": "BRANCH-001",
    "readingKey": "2026-08-12T09:00:00.000Z#READING-001"
  }
}
```

Add a few more readings

This section is just adding more parts to the database similar to the first test.

```
1 {
2   "action" : "create",
3   "branchId" : "BRANCH-001",
4   "readingId" : "READING-002",
5   "recordedAt" : "2026-08-10T09:00:00.000Z",
6   "minTempC" : 3.5,
7   "maxTempC" : 7.2,
8   "recordedBy" : "Umar"
9 }
```

```

1 {
2   "action" : "create",
3   "branchId" : "BRANCH-001",
4   "readingId" : "READING-003",
5   "recordedAt" : "2026-08-11T09:00:00.000Z",
6   "minTempC" : 1.9,
7   "maxTempC" : 7,
8   "recordedBy" : "Umar"
9 }

```

Result = this gives you three total readings in your database; this can be seen in the table below.

☐	branchId (String)	readingKey (String)	maxTempC	minTempC	readingId	recordedAt	recordedBy	status
☐	BRANCH-001	2026-08-10T09:00:00.0...	7.2	3.5	READING-002	2026-08-10...	Umar	ok
☐	BRANCH-001	2026-08-11T09:00:00.0...	7	1.9	READING-003	2026-08-11...	Umar	excursion
☐	BRANCH-001	2026-08-12T09:00:00.0...	7	3	READING-001	2026-08-12...	Umar	ok

QUERY all readings for BRANCH-001

Test script: **QueryBranch**

```

1 {
2   "action" : "queryBranch",
3   "branchId" : "BRANCH-001"
4 }

```

Result = This command asked 'give me the items belonging to BRANCH-001', and you can see the result below. It gave us all three readings.

```

Response:
{
  "readings": [
    {
      "recordedBy": "Umar",
      "readingId": "READING-001",
      "status": "ok",
      "minTempC": 3,
      "maxTempC": 7,
      "recordedAt": "2026-08-12T09:00:00.000Z",
      "branchId": "BRANCH-001",
      "readingKey": "2026-08-12T09:00:00.000ZREADING-001"
    },
    {
      "recordedBy": "Umar",
      "readingId": "READING-003",
      "status": "excursion",
      "minTempC": 1.9,
      "maxTempC": 7,
      "recordedAt": "2026-08-11T09:00:00.000Z",
      "branchId": "BRANCH-001",
      "readingKey": "2026-08-11T09:00:00.000ZREADING-003"
    },
    {
      "recordedBy": "Umar",
      "readingId": "READING-002",
      "status": "ok",
      "minTempC": 3.5,
      "maxTempC": 7.2,

```

QUERY between two dates

Test script: **QueryDateRange**

```

1 {
2   "action" : "queryDateRange",
3   "branchId" : "BRANCH-001",
4   "from" : "2026-08-10T00:00:00.000Z",
5   "to" : "2026-08-11T23:59:59.999Z"
6 }

```

Result = this returns two of the readings that are between from: "2026-08-10T00:00:00.000Z" and to: "2026-08-11T23:59:59.999Z".

```
PROBLEMS OUTPUT CODE REFERENCE LOG TERMINAL
Test Event Name: QueryDateRange
Response:
{
  "readings": [
    {
      "recordedBy": "Umar",
      "readingId": "READING-001",
      "status": "excursion",
      "minTempC": 1.9,
      "maxTempC": 7,
      "recordedAt": "2026-08-11T09:00:00.000Z",
      "branchId": "BRANCH-001",
      "readingKey": "2026-08-11T09:00:00.000Z#READING-001"
    },
    {
      "recordedBy": "Umar",
      "readingId": "READING-002",
      "status": "ok",
      "minTempC": 3.5,
      "maxTempC": 7.2,
      "recordedAt": "2026-08-10T09:00:00.000Z",
      "branchId": "BRANCH-001",
      "readingKey": "2026-08-10T09:00:00.000Z#READING-002"
    }
  ]
}
```

UPDATE a reading

Test script: **UpdateReading**

```
1 {
2   "action" : "update",
3   "branchId" : "BRANCH-001",
4   "readingKey" : "2026-08-12T09:00:00.000Z#READING-001",
5   "recordedBy" : "Umar Hussain"
6 }
```

Result =

```
JS index.mjs x
JS index.mjs > ...
1 import { DynamoDBClient } from "@aws-sdk/client-dynamodb"; Amazon Q Tip 1/3: Start typing to get suggestions ([ESC] to exit)

PROBLEMS OUTPUT CODE REFERENCE LOG TERMINAL
Status: Succeeded
Test Event Name: UpdateReading
Response:
{
  "message": "Reading updated",
  "item": {
    "readingKey": "2026-08-12T09:00:00.000Z#READING-001",
    "minTempC": 3,
    "recordedBy": "Umar Hussain",
    "status": "ok",
    "recordedAt": "2026-08-12T09:00:00.000Z",
    "readingId": "READING-001",
    "branchId": "BRANCH-001",
    "maxTempC": 7
  }
}

The area below shows the last 4 KB of the execution log.

Function Logs:
START RequestId: 6926ee6f-d8c6-4187-8a18-bc33839b08be Version: $LATEST
2026-08-13T11:13:15.398Z 6926ee6f-d8c6-4187-8a18-bc33839b08be INFO Request received {
```

DELETE a reading

Test script: **DeleteReading**

! Warning

This is done at the last step because once it is **deleted it is gone**.

```
1 {
2   "action" : "delete",
3   "branchId" : "BRANCH-001",
4   "readingKey" : "2026-08-12T09:00:00.000Z#READING-001"
5 }
```

Result = the result should just show message: 'reading deleted' — this is shown below:

Status: Succeeded
Test Event Name: DeleteReading

Response:

```
{  
  "message": "Reading deleted"  
}
```

The area below shows the last 4 KB of the execution log.

⚠ Questions / Queries / Pushback

No worries for this section so far.